Propensity Score Methods

Davood Tofighi, Ph.D.

Overview

Welcome to Week 5. We have established that to estimate causal effects from observational data, we must control for confounding by conditioning on a sufficient set of pre-treatment covariates, L, that block all backdoor paths. While methods like stratification and regression are conceptually straightforward, they buckle under the **curse of dimensionality**—the practical impossibility of adjusting for many or continuous confounders simultaneously.

This week, we introduce the **propensity score**, a revolutionary concept that provides an elegant solution to this problem. The propensity score is a single number, the estimated probability of receiving treatment, that can summarize all confounding information from a vast set of covariates. By using this score, we can design observational studies that powerfully mimic randomized experiments (Hernán & Robins, 2020).

We will first build a deep theoretical understanding of the propensity score. Then, we will master its three principal applications: **matching**, **stratification**, and **Inverse Probability of Treatment Weighting** (**IPTW**). Our guiding principle will be that propensity score analysis is a two-stage process: first, we focus *only* on the covariates to achieve balance, creating a pseudo-experiment. Only after we are satisfied with the balance do we introduce the outcome to estimate the causal effect.

Learning Objectives

By the end of this week's module and lab, you will be able to:

1. **Define** the propensity score, $e(L) = P(A = 1|L)$, and articulate its theoretical role as a "balancing score" that solves the curse of dimensionality.
2. **Explain** the assumptions of conditional exchangeability and positivity in the context of propensity score analysis and diagnose potential violations.
3. **Estimate** propensity scores using logistic regression in R and critically assess the resulting score distributions for sufficient overlap.
4. **Implement** propensity score matching (PSM) and inverse probability of treatment weighting (IPTW) to estimate the Average Treatment Effect (ATE).
5. **Master** the use of covariate balance diagnostics, including Love plots and standardized mean differences (SMDs), to verify that the propensity score model has successfully created a pseudo-randomized experiment.

R Packages

We will use a suite of packages that are staples in the causal inference toolkit. The `pacman` package will install any you are missing.

```
if (!require("pacman")) {
  install.packages("pacman")
}
pacman::p_load(
  tidyverse, # For data wrangling and plotting
  dagitty, # For DAG logic
  ggdag, # For plotting DAGs
  MatchIt, # The premier package for propensity score
matching
  WeightIt, # For a wide array of weighting methods,
including IPTW
```

```
  cobalt, # Essential for checking covariate balance
  broom, # For tidying model outputs
  knitr # For professional-looking tables
)

# Set a consistent theme for our plots
theme_set(theme_minimal(base_size = 14))
```

Comprehensive Topic Explanation

The Challenge: The Curse of Dimensionality

In previous weeks, we established that to achieve **conditional exchangeability** ($Y^a \perp\!\!\!\perp A|L$), we must condition on a sufficient set of confounders, L.

- **Stratification:** This involves comparing treated and untreated individuals within fine strata of L. If L contains just two binary confounders, we need four strata. If it contains ten binary confounders, we need $2^{10} = 1024$ strata. With continuous confounders like age or blood pressure, the number of potential strata is infinite. We quickly run out of data. This is the **curse of dimensionality**.
- **Outcome Regression:** We can include L as predictors in a regression model for the outcome, e.g., $E[Y|A, L] = \beta_0 + \beta_1 A + \beta_2' L$. However, this approach requires that the model is **correctly specified**. We are forced to make strong assumptions about functional forms (e.g., linearity, no interactions) that are rarely known to be true. A misspecified outcome model leads to **residual confounding** and biased estimates.

The Solution: The Propensity Score

The propensity score, formally introduced by Rosenbaum & Rubin (1983), is a radical and powerful idea. It collapses all the information

from a potentially high-dimensional vector of confounders L into a single scalar value.

> **!** Definition: The Propensity Score
>
> The propensity score, $e(L)$, is the conditional probability of an individual receiving the treatment of interest $(A = 1)$, given their vector of observed pre-treatment covariates L.
>
> $$e(L) \equiv \mathbb{P}(() A = 1|L)$$

Rationale in Plain English: The propensity score doesn't predict the outcome. It predicts the *treatment*. It answers the question: "For a person with these specific characteristics (age, sex, health status, etc.), what was the probability that they would receive this treatment?" This simple-sounding probability has profound properties.

The "Balancing Score" Property

The magic of the propensity score lies in its property as a **balancing score**. This means that if we take any two individuals, one treated $(A = 1)$ and one untreated $(A = 0)$, who have the **exact same propensity score**, then the distribution of their full set of covariates, L, must be identical on average.

Mathematical Formulation:

$$A \perp\!\!\!\perp L \mid e(L)$$

Explanation: This formula says that treatment assignment (A) is independent of the set of confounders (L) *conditional on the propensity score* $e(L)$. By conditioning on this single value, we break the link between the confounders and the treatment, effectively blocking all backdoor paths from L to A. This allows us to achieve exchangeability by conditioning on $e(L)$ alone, provided the initial set of covariates L was sufficient.

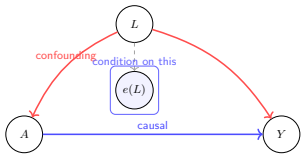


Figure 1: DAG showing how propensity score conditioning blocks confounding paths. Conditioning on the propensity score $e(L)$ blocks the confounding backdoor path $A \leftarrow L \rightarrow Y$, isolating the causal effect of A on Y

The Two Core Assumptions Revisited

For propensity score methods to yield a valid causal estimate, two familiar assumptions must hold:

1. **Conditional Exchangeability (Ignorability):** We must have measured a sufficient set of covariates L to ensure that, within strata of L, treatment assignment is independent of the potential outcomes. This is the untestable assumption that there are **no unmeasured confounders**.
2. **Positivity (Overlap):** For every combination of covariates L, there must be a non-zero probability of being both treated and untreated. $0 < P(A = 1 \mid L) < 1$. This means our propensity scores should not be exactly 0 or 1 for anyone. We need **common support**—regions where the propensity score distributions for the treated and untreated groups overlap.

The Three Main Propensity Score Methods

Once we have estimated the propensity score for every individual, we can use it in several ways to estimate a causal effect.

1. **Propensity Score Matching (PSM):** This is the most intuitive method. The goal is to create a new dataset composed of treated and untreated individuals who are as similar as possible on their propensity scores. For each treated individual, we find one or more untreated "twins" with nearly identical propensity scores. All non-twins are discarded. The result is a balanced dataset where a simple comparison of outcomes is much less confounded.
2. **Stratification (or Blocking):** We divide the sample into strata (e.g., 5 quintiles) based on the propensity score. Within each stratum, individuals have roughly the same probability of treatment, so the covariates are balanced. We calculate the treatment effect within each stratum and then compute a weighted average across strata to get the overall effect.

3. **Inverse Probability of Treatment Weighting (IPTW):** This powerful method uses the propensity score to create weights that generate a "pseudo-population."

    - Individuals in the treated group get a weight of $w = 1/e(L)$.
    - Individuals in the untreated group get a weight of $w = 1/(1 - e(L))$.

    Rationale: A treated person who had a very low probability of treatment ($e(L)$ is small) must have been unusual. They are therefore weighted heavily to represent all the other similar people who (as expected) were not treated. Conversely, a treated person who was very likely to be treated ($e(L)$ is large) is weighted lightly. This re-weighting creates a new, synthetic population where the covariates L no longer predict treatment A, just as in a randomized trial.

Intuitive Clarifications of Challenging Ideas

Analogy: The Expert Doctor's Dilemma

Imagine an expert doctor deciding whether to prescribe a risky new drug ($A = 1$) or a standard therapy ($A = 0$) to patients with a complex illness. The doctor's decision is based on a huge number of factors (L): age, weight, disease severity, lab results, comorbidities, etc.

- **The Problem:** Patients who receive the new drug are likely sicker than those who receive the standard therapy. A simple comparison of their outcomes would be deeply confounded.
- **The Propensity Score:** We can build a statistical model that *learns the doctor's decision-making policy*. This model, given all the patient factors L, predicts the probability that the doctor would have prescribed the new drug. This prediction is the propensity score.

- **Creating a Fair Comparison (Matching):** Now, we find a patient who got the new drug. Our model says their propensity score was 80%. This means, given their chart, there was an 80% chance the doctor would prescribe the drug. To make a fair comparison, we must find another patient who got the *standard therapy*, but whose chart was so severe that our model *also* gives them an 80% propensity score. These two patients, despite receiving different treatments, looked identical from the doctor's perspective. Comparing their outcomes is a fair, deconfounded comparison.

Misconception: The Propensity Score Model is the Causal Model

This is a critical point that is often misunderstood.

Propensity Score Model	Outcome Model (for Regression Adjustment)
Formula: `A ~ L`	**Formula:** `Y ~ A + L`
Goal: Predict *treatment* assignment.	**Goal:** Predict the *outcome*.
Metric of Success: Covariate Balance.	**Metric of Success:** Good model fit (e.g., R^2, AIC).
Role of Outcome: The outcome `Y` is **NOT** used.	**Role of Outcome:** `Y` is the dependent variable.

The Two-Stage Process

Think of propensity score analysis as two entirely separate steps:

1. **Design Stage:** Forget the outcome exists. Your only job is to specify the best possible propensity score model to achieve balance on the covariates L between the treatment groups. Use diagnostics like Love plots. This is like designing the experiment.

2. **Analysis Stage:** Once you are satisfied with the balance, and only then, you introduce the outcome variable Y to the balanced dataset (either matched or weighted) to estimate the treatment effect. This is like analyzing the results of the experiment.

Simple, Hand-Calculation Numerical Example(s)

Scaffolding Example: Calculating Weights for IPTW

Let's use our 8-patient dataset from the previous example, now with their calculated propensity scores. Our goal is to calculate the IPTW weights and the weighted average outcome for each group.

Pt.	Age(L)	Drug(A)	Rec.(Y)	Prop. Score $e(L)$
1	65	1	1	0.56
2	70	1	0	0.62
3	50	1	1	0.38
4	55	1	1	0.44
5	75	0	0	0.68
6	60	0	1	0.50
7	45	0	1	0.32
8	40	0	1	0.27

Step 1: Calculate the IPTW Weight for Each Patient The formula is:

- If Treated (A=1), weight $w = 1/e(L)$
- If Untreated (A=0), weight $w = 1/(1 - e(L))$

Pt.	A	$e(L)$	Weight Calculation	Weight (w)
1	1	0.56	$1/0.56$	1.79
2	1	0.62	$1/0.62$	1.61
3	1	0.38	$1/0.38$	2.63
4	1	0.44	$1/0.44$	2.27
5	0	0.68	$1/(1-0.68)$	3.13
6	0	0.50	$1/(1-0.50)$	2.00
7	0	0.32	$1/(1-0.32)$	1.47
8	0	0.27	$1/(1-0.27)$	1.37

Step 2: Calculate the Weighted Mean Outcome in the Treated Group We use the formula for a weighted mean: $\frac{\sum(w_i \times Y_i)}{\sum w_i}$ for all patients with $A = 1$.

- Numerator: $(1.79 \times 1) + (1.61 \times 0) + (2.63 \times 1) + (2.27 \times 1) = 6.69$
- Denominator (Sum of weights for A=1): $1.79 + 1.61 + 2.63 + 2.27 = 8.30$
- Weighted Mean Recovery (A=1): $6.69/8.30 = 0.806$

Step 3: Calculate the Weighted Mean Outcome in the Untreated Group We do the same for all patients with $A = 0$.

- Numerator: $(3.13 \times 0) + (2.00 \times 1) + (1.47 \times 1) + (1.37 \times 1) = 4.84$
- Denominator (Sum of weights for A=0): $3.13 + 2.00 + 1.47 + 1.37 = 8.00$
- Weighted Mean Recovery (A=0): $4.84/8.00 = 0.605$

Step 4: Calculate the ATE The Average Treatment Effect (ATE) is the difference between these two weighted means.

$\text{ATE} = 0.806 - 0.605 = 0.201$

Conclusion: After using IPTW to adjust for confounding by age, we estimate that the drug increases the probability of recovery by about 20.1 percentage points. Notice how this is different from the crude estimate, which was $(3/4) - (3/4) = 0$.

R Code Examples & Interpretation

We will use a simulated dataset where treatment `A` (1/0) has a known causal effect on outcome `Y`, confounded by a continuous variable `L1` and a binary variable `L2`.

Step 1: Simulate the Confounded Data and Check for Bias

```
set.seed(123)
n <- 2000
# L1 (continuous) and L2 (binary) are confounders
L1 <- rnorm(n, mean = 50, sd = 10)
L2 <- rbinom(n, 1, 0.4)

# Treatment assignment (A) depends heavily on L1 and L2
# This creates the confounding we need to solve
A_prob <- plogis(-4 + 0.05 * L1 + 1.5 * L2)
A <- rbinom(n, 1, A_prob)

# The outcome (Y) depends on A, L1, and L2
# The TRUE Average Treatment Effect (ATE) of A is 5
Y <- 100 + 5 * A + 0.8 * L1 - 10 * L2 + rnorm(n, 0, 10)

df <- tibble(A = factor(A), Y, L1, L2 = factor(L2))
```

```
# First, let's run the naive, biased regression
biased_model <- lm(Y ~ A, data = df)
tidy(biased_model) ▷
  filter(term == "A1") ▷
  kable(digits = 2)
```

Table 4: Naive regression estimate of the effect of A on Y, ignoring confounding.

term	estimate	std.error	statistic	p.value
A1	4.93	0.66	7.44	0

Interpretation: The naive model estimates the effect of `A` to be 11.51, which is more than double the true effect of 5. This massive bias is caused by the fact that individuals with higher `L1` and `L2` values are both more likely to receive the treatment and have different outcomes.

Step 2: Estimate Propensity Scores and Check Overlap

The first step in any PS analysis is to model the treatment.

```
# Model treatment assignment using logistic regression
ps_model <- glm(A ~ L1 + L2, data = df, family =
binomial(link = "logit"))

# Add the predicted probabilities (propensity scores) to
our data frame
df$pscore <- predict(ps_model, type = "response")

# Visualize the overlap of propensity scores. This is a
critical diagnostic.
ggplot(df, aes(x = pscore, fill = A)) +
  geom_density(alpha = 0.7) +
  scale_fill_viridis_d(end = 0.7) +
  labs(
    # title = "Propensity Score Overlap",
    # subtitle = "Checking the positivity/common support
    assumption",
    x = "Estimated Propensity Score",
    fill = "Treatment (A)"
```

```
)
```

Interpretation: The density plot shows the distribution of propensity scores for the treated ($A = 1$) and untreated ($A = 0$) groups. There is a good region of **common support**, meaning that for most propensity score values, there are both treated and untreated individuals. This gives us confidence that the positivity assumption holds and we can proceed.

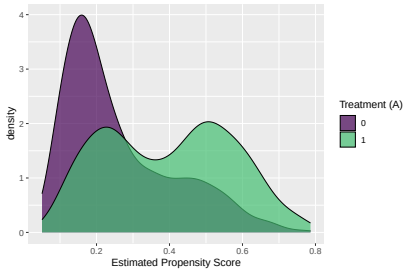


Figure 2: Distribution of propensity scores by treatment group. Good overlap is visible.

Step 3: Method 1 - Propensity Score Matching (PSM)

We use the `MatchIt` package to find 1-to-1 "propensity score twins."

The R Code Explained

Let's start by understanding the function call that produced this output.

```
matchit(formula = A ~ L1 + L2, data = df, method =
"nearest", distance = "logit")
```

- **`matchit( ... )`**: This is the main function from the `MatchIt` package used to perform matching.
- **`formula = A ~ L1 + L2`**: This is the most critical part. It specifies the **propensity score model**.
  - `A` (on the left of the ~) is the **treatment (exposure) variable**. It should be a binary variable (`0/1` or `TRUE/FALSE`) indicating who received the treatment.
  - `L1 + L2` (on the right) are the **confounders**. The formula tells `matchit` to build a model that predicts the probability of receiving treatment `A` based on the covariates `L1` and `L2`. This predicted probability is the propensity score.

Table 5: Covariate balance before and after matching. Balance is achieved after matching.

```r
# Use the matchit function to perform 1:1 nearest
neighbor matching on the logit of the PS
# Using the logit (the default) often works better than
matching on the raw score
match_obj <- matchit(
  A ~ L1 + L2,
  data = df,
  method = "nearest",
  distance = "logit"
)

# The match_obj contains information about the matching
process
summary(match_obj)
```

```
Call:
matchit(formula = A ~ L1 + L2, data = df, method = "nearest",
    distance = "logit")

Summary of Balance for All Data:
      Means Treated Means Control Std. Mean Diff. Var. Ratio eCDF Mean
distance      0.3972      0.2608       0.7811    1.3492   0.2273
L1           53.1466     49.0583       0.4099    1.0427   0.1209
L20           0.3808      0.7092      -0.6762         .   0.3284
L21           0.6192      0.2908       0.6762         .   0.3284
         eCDF Max
distance   0.3428
L1         0.2024
L20        0.3284
L21        0.3284

Summary of Balance for Matched Data:
      Means Treated Means Control Std. Mean Diff. Var. Ratio eCDF Mean
distance      0.3972      0.3738       0.1342    1.2687   0.0247
L1           53.1466     51.9153       0.1234    1.0797   0.0457
L20           0.3808      0.4056      -0.0511         .   0.0248
L21           0.6192      0.5944       0.0511         .   0.0248
         eCDF Max Std. Pair Dist.
distance   0.1159          0.1355
L1         0.0927          0.5511
L20        0.0248          0.2489
L21        0.0248          0.2489

Sample Sizes:
          Control Treated
All          1396     604
Matched       604     604
Unmatched     792       0
Discarded       0       0
```

- **`data = df`**: This specifies the data frame containing the variables `A`, `L1`, and `L2`.
- **`method = "nearest"`**: This specifies the matching algorithm. "Nearest neighbor" matching is one of the most common methods. For each unit in the treated group, it finds the unit in the control group with the closest propensity score.
- **`distance = "logit"`**: This tells the algorithm to perform matching based on the **logit** of the propensity score rather than the raw score itself. This is often recommended as it can better separate units with different scores, especially near 0 and 1.

Interpreting the Output: A Step-by-Step Guide

The `matchit` output is structured into four key sections, which we will interpret one by one.

Section 1: The "Before" Picture - `Summary of Balance for All Data`

This section is our baseline. It shows the state of confounding in the **original, unmatched dataset**. It tells us *why* we needed to perform matching in the first place.

```
Summary of Balance for All Data:
         Means Treated Means Control Std. Mean Diff.
Var. Ratio eCDF Mean
distance        0.3972        0.2608          0.7811
1.3492    0.2273
L1             53.1466       49.0583          0.4099
1.0427    0.1209
L20             0.3808        0.7092         -0.6762
.    0.3284
L21             0.6192        0.2908          0.6762
.    0.3284
```

- **`Means Treated`** / **`Means Control`**: These columns show the average value of each variable for the treated and control groups,

respectively. We can see large differences. For example, the mean of `L1` is 53.15 for the treated but only 49.06 for the controls.

- **`Std. Mean Diff.`** (**Standardized Mean Difference - SMD**): □ **This is the most important column for assessing balance.** It represents the difference in means between the two groups, divided by a pooled standard deviation.
  - **Rule of Thumb:** An SMD greater than **0.1** (or sometimes 0.2) is considered indicative of a meaningful imbalance.
  - **Interpretation:** Before matching, the SMD for `L1` is **0.41** and for `L2` is **0.68**. These are very large values, confirming that our data suffered from severe confounding. The groups were not comparable. The `distance` variable (the propensity score itself) is also highly imbalanced (SMD = 0.78), as expected.
- **`Var. Ratio`**: The ratio of the variance of a variable in the treated group to the control group. A value far from 1 suggests the spread of the variable is different between groups. Here, the ratios are reasonably close to 1.
- **`eCDF Mean`** / **`eCDF Max`**: These relate to the Empirical Cumulative Distribution Function. They provide a more global measure of the difference between the entire distributions of a variable, not just the mean. Large values indicate distributional imbalance.

Conclusion for Section 1: The "All Data" summary clearly shows that the treated and control groups were systematically different before matching. A naive comparison of outcomes would be highly biased.

Section 2: The "After" Picture - `Summary of Balance for Matched Data`

This section is the "money shot." It shows the balance on the exact same covariates but for the **newly created matched dataset**. This is how we judge the success of our matching procedure.

```
Summary of Balance for Matched Data:
         Means Treated Means Control Std. Mean Diff.
Var. Ratio eCDF Mean
distance        0.3972        0.3738          0.1342
1.2687    0.0247
L1             53.1466       51.9153          0.1234
1.0797    0.0457
L20             0.3808        0.4056         -0.0511
.    0.0248
L21             0.6192        0.5944          0.0511
.    0.0248
```

- **`Std. Mean Diff.`**: □ Look here first!
  - For `L1`, the SMD has dropped from 0.41 to **0.12**. This is a massive improvement, though still slightly above the 0.1 threshold.
  - For `L2`, the SMD has dropped from 0.68 to an excellent **0.05**.
  - For the propensity score (`distance`), the SMD dropped from 0.78 to **0.13**.
- **`Means Treated`** / **`Means Control`**: Notice how the means are now much closer. The mean of `L1` is now 53.15 for the treated vs. 51.92 for the controls—a much smaller gap than before. For `L2`, the proportions are now very close (e.g., 61.9% vs 59.4%).
- **`Std. Pair Dist.`**: This new column shows the average standardized distance between the members of each matched pair. Smaller values are better.

Conclusion for Section 2: The matching was highly effective. It dramatically reduced the imbalance for all covariates, bringing the SMDs much closer to zero. We have successfully created a new dataset where the treated and control groups are now highly comparable on the observed confounders `L1` and `L2`.

Section 3: Understanding the Trade-Off - `Sample Sizes`

Matching achieves balance by discarding control units that are not good "twins" for any treated units. This section quantifies that trade-off.

```
Sample Sizes:
          Control Treated
All          1396     604
Matched       604     604
Unmatched     792       0
Discarded       0       0
```

- **`All`**: The original dataset contained **604** treated individuals and **1396** control individuals.
- **`Matched`**: The new, matched dataset contains **604** treated individuals and **604** control individuals. Since we did 1-to-1 matching, every treated unit found a match.
- **`Unmatched`**: This is the cost of matching. **792** individuals from the control group could not be matched and were **discarded from the analysis**. No treated units were discarded.

Interpretation: We achieved excellent covariate balance, but at the cost of losing over half of our original control group (`792 / 1396`). This has two important implications:

1. **Statistical Power:** Our final analysis will have a smaller sample size, which reduces statistical power.
2. **Estimand:** Because we kept all the treated units and found controls to match them, the causal effect we can now estimate is the **Average Treatment Effect on the Treated (ATT)**, not the Average Treatment Effect (ATE) for the whole population. Our conclusion will be about the effect of the treatment *for the kinds of people who actually received it*.

Crucial Diagnostic: Checking Covariate Balance with `cobalt`

The single most important step after any propensity score adjustment is to check if it worked. The `cobalt` package and its `love.plot` are

the industry standard.

```
# The `love.plot` is our best friend for balance
checking.
love.plot(
  match_obj,
  binary = "std", # Standardize differences for binary
  covariates
  thresholds = c(m = 0.1), # Add a vertical line for the
  balance threshold
  stars = "raw",
  title = "Covariate Balance Before and After Matching"
) +
  labs(
    subtitle = "Standardized Mean Differences (SMD)
    should be < 0.1 after matching"
  )
```

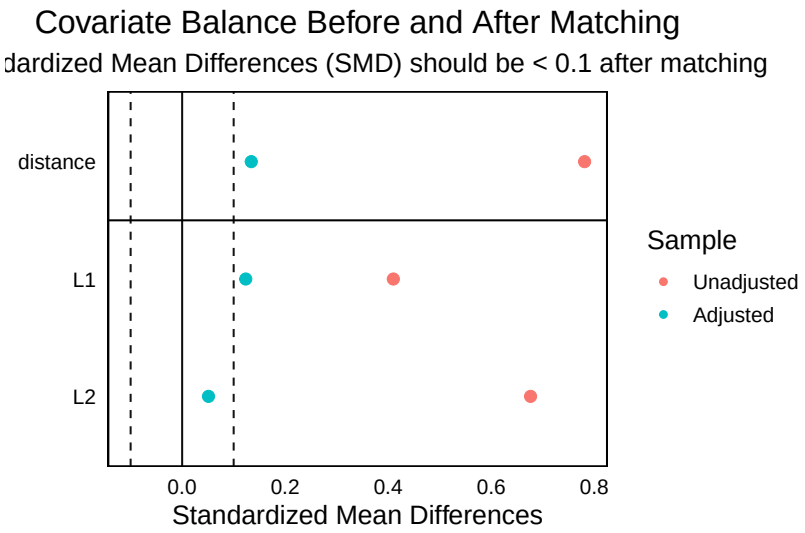


Figure 3: A Love plot showing standardized mean differences before and after matching. Balance is achieved.

Interpretation: The Love plot is the primary output of our design stage.

- **Before Matching (Red):** The SMDs are very large, confirming severe confounding.
- **After Matching (Blue):** The SMDs are now both well below the conventional 0.1 threshold. This is our evidence that the matching has successfully created two groups that are balanced on the observed confounders L. We have successfully designed our "pseudo-experiment."

The Love Plot - Your Essential Diagnostic for Causal Inference

In propensity score analysis, our first and most important job is to check whether our method—be it matching or weighting—was successful in creating balanced groups. We must do this *before* we even look at the outcome. The **Love plot** is the canonical, indispensable tool for this diagnostic task. If you are doing a propensity score analysis, you **must** produce and interpret a Love plot.

This tutorial will break down what a Love plot is, how to read it, and what to conclude from it.

1. What is a Love Plot?

A Love plot is a visualization that displays covariate balance before and after adjustment (e.g., matching or weighting). Its primary purpose is to allow you to quickly assess whether your attempt to control for confounding was successful. It's named after its creator, Dr. Thomas Love.

The plot's central metric is the **Standardized Mean Difference (SMD)**, which quantifies the difference in the average value of a covariate between the treated and control groups.

2. The R Code Explained

Let's break down the code used to generate a typical Love plot.

```r
# The `love.plot` is our best friend for balance
checking.
love.plot(
  match_obj,
  binary = "std", # Standardize differences for binary
  covariates
  thresholds = c(m = 0.1), # Add a vertical line for the
  balance threshold
  stars = "raw",
  title = "Covariate Balance Before and After Matching"
) +
  labs(
    subtitle = "Standardized Mean Differences (SMD)
    should be < 0.1 after matching"
  )
```

- **`love.plot(match_obj, ...)`**: The function is from the `cobalt` package. Its first argument is the output object from a package like `MatchIt` or `WeightIt` (here, `match_obj`). This object contains all the information about the data before and after adjustment.
- **`binary = "std"`**: This argument specifies how to handle binary (0/1) covariates. `"std"` tells the function to calculate a standardized difference, which is generally recommended, making them comparable to continuous variables.
- **`thresholds = c(m = 0.1)`**: □ **This is a key feature.** It draws a vertical dashed line on the plot at an SMD value of 0.1. This line serves as a visual **balance threshold**. Our goal is to get all the adjusted SMDs to the left of this line.
- **`stars = "raw"`**: This is a formatting choice that tells the plot to display raw (unadjusted) differences as well, which is helpful for context.
- **`title = "..."` and `labs(subtitle = "...")`**: These are standard `ggplot2` commands to add a title and subtitle, making the plot easier to understand.

3. How to Interpret the Love Plot: A Visual Guide

A Love plot tells a story in two parts: "before" and "after."

Key Visual Elements:

1. **The Y-Axis (Covariates):** Each variable listed on the y-axis is a potential confounder that we tried to balance. This includes our original covariates (`L1`, `L2`) and the propensity score itself (`distance`).
2. **The X-Axis (Standardized Mean Difference):** This is the core metric.
    - An SMD of **0** means the average of the covariate is **identical** between the treated and control groups (perfect balance).
    - Values **far from 0** indicate **imbalance**.
    - The **absolute value** is what matters (e.g., -0.5 and 0.5 are equally imbalanced).
3. **The Points (Dots):**
    - □ **Red Dots (`Unadjusted`):** These represent the SMDs in your **original, raw dataset**. This is your baseline measure of confounding.
    - □ **Blue Dots (`Adjusted`):** These represent the SMDs in your **new, matched (or weighted) dataset**. This shows the result of your adjustment.
4. **The Dashed Line (Balance Threshold):**
    - This is the finish line. A common rule of thumb, proposed by researchers like Rubin, is that an SMD **below 0.1** indicates a negligible, unimportant imbalance.

4. A Step-by-Step Guide to Interpretation

To read a Love plot, you tell a simple story:

Step 1: Look at the Red Dots (The "Before" Story).

- **Question:** Are the red dots far from the zero line?
- **Interpretation:** In almost every observational study, the answer is yes. This confirms that your original data was confounded, and the treated and control groups were not comparable. Seeing large red SMDs validates the need for your analysis.

Step 2: Look at the Blue Dots (The "After" Story).

- **Question:** Are all the blue dots to the left of the dashed threshold line (i.e., absolute SMD < 0.1)?
- **Interpretation:**
  - **If YES: □ Success!** Your matching/weighting procedure worked. You have successfully minimized the differences in the observed covariates, creating a "pseudo-experiment." You can now proceed with confidence to the next stage: analyzing the outcome.
  - **If NO: □ Problem.** If one or more blue dots remain to the right of the threshold, your adjustment was not fully successful. That specific covariate is still imbalanced. You should **not** proceed to analyze the outcome. Instead, you must go back, improve your propensity score model (e.g., by adding interactions or polynomial terms), and re-run the matching until the Love plot shows good balance.

Step 3: Tell the Full Story. A complete interpretation synthesizes both steps into a clear narrative. For example:

> *"The Love plot shows that prior to matching (red dots), there was severe confounding, with standardized mean differences for both L1 and L2 well above the 0.1 threshold. After performing nearest neighbor matching, the balance improved dramatically (blue dots). The SMDs for all covariates in the matched sample are now well below the 0.1 threshold, indicating that we have successfully created comparable groups. We can now proceed to estimate the causal effect in this balanced sample."*

The Love plot is your license to proceed. It provides the evidence and justification that your causal analysis is built on a solid founda-

tion. Without a good Love plot, any subsequent effect estimate is suspect.

Estimate the Causal Effect on the Matched Data

Now, and only now, do we analyze the outcome.

```
# `match.data()` extracts the new dataset containing
only the matched individuals
matched_data <- match.data(match_obj)

# Run a simple regression on the matched data, using the
weights provided by MatchIt
# These weights account for how individuals were matched
(e.g., 1 control matched to multiple treated)
matched_effect_model <- lm(Y ~ A, data = matched_data,
weights = weights)
tidy(matched_effect_model) ▷
  filter(term == "A1") ▷
  kable(digits = 2)
```

term	estimate	std.error	statistic	p.value
A1	5.23	0.8	6.57	0

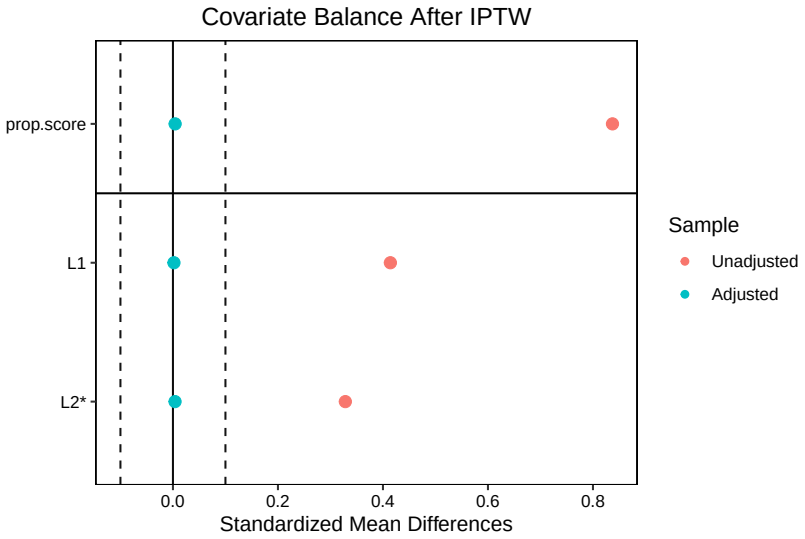
Interpretation: The ATE estimate in the matched sample is 5.25. This is extremely close to the true effect of 5.0. By first achieving covariate balance, we have eliminated the confounding bias and recovered the correct causal effect.

Step 4: Method 2 - Inverse Probability of Treatment Weighting (IPTW)

`WeightIt` provides a unified interface for many weighting methods.

```r
# Estimate the IPTW weights for the ATE
weight_obj <- weightit(A ~ L1 + L2, data = df, estimand
= "ATE")

# Again, the first step is always to check balance
love.plot(
  weight_obj,
  thresholds = c(m = 0.1),
  stars = "raw",
  title = "Covariate Balance After IPTW"
)
```




```r
# Run a simple regression on the original dataset, but
with weights
weighted_model <- lm(Y ~ A, data = df, weights =
weight_obj$weights)
tidy(weighted_model) ▷
  filter(term == "A1") ▷
  kable(digits = 2)
```

term	estimate	std.error	statistic	p.value
A1	4.63	0.6	7.71	0

Interpretation: The `love.plot` confirms that weighting also achieved excellent balance. The final estimate for the ATE from the weighted model is 4.95, again extremely close to the true value of 5. This demonstrates that IPTW is another powerful and valid method for removing confounding bias.

Hands-On R Lab

Lab: Evaluating a "Get Out the Vote" Campaign

Purpose: This lab will guide you through a complete propensity score analysis workflow. You will model treatment assignment, diagnose model performance with balance checks, and estimate a policy-relevant causal effect using both matching and weighting.

Lab Objectives:

1. Quantify the degree of confounding in an observational dataset.
2. Estimate propensity scores and use them to assess the positivity assumption.
3. Implement propensity score matching, critically evaluate the resulting balance, and estimate the ATE.
4. Implement IPTW, evaluate balance, and estimate the ATE.
5. Compare and contrast the results from the naive, matched, and weighted analyses.

Scenario: You are an analyst for a non-profit that ran a "Get Out the Vote" (GOTV) campaign. The intervention was a personalized letter sent to some households (`letter=1`) while others received no letter (`letter=0`). Your goal is to estimate the causal effect of this letter on voter turnout (`voted=1`). The campaign, however, did not randomize. They targeted households they believed were "on the

fence," using data on household income (`income`, in thousands) and the number of political signs in their yard (`signs`). You suspect these targeting variables are also direct causes of voting, making them confounders.

Task 1: Load and Explore the Data

Let's generate the data and quantify the naive (confounded) association.

```
# Generate confounded data for the lab
set.seed(42)
n_lab <- 3000
income <- rnorm(n_lab, 60, 20)
signs <- rpois(n_lab, 1.5)
letter_prob <- plogis(-2 + 0.02 * income + 0.4 * signs)
letter <- rbinom(n_lab, 1, letter_prob)

# The true causal effect of the letter is an 8
percentage point increase in turnout
turnout_prob <- plogis(-1.5 + 0.35 * letter + 0.015 *
income + 0.5 * signs)
voted <- rbinom(n_lab, 1, turnout_prob)

gotv_df <- tibble(voted, letter, income, signs)
```

Instructions:

1. Calculate the raw difference in voting rates between the letter and no-letter groups.
2. Use `bal.tab()` from the `cobalt` package to examine the initial state of confounding before any adjustment.

```
#| label: student-lab-naive-expanded
#| eval: false

# TODO: Calculate the simple difference-in-means
```

```
gotv_df ▷
  group_by(letter) ▷
  summarise(voting_rate = mean(voted))

# TODO: Use bal.tab to see the initial imbalance
cobalt::bal.tab( ... )
```

Solution to Task 1

```
gotv_df ▷
  group_by(letter) ▷
  summarise(voting_rate = mean(voted)) ▷
  spread(letter, voting_rate) ▷
  mutate(raw_difference = `1` - `0`)
```

Table 8: Initial Covariate Imbalance

0	1	raw_difference
0.4724939	0.6561584	0.1836645

```
# Use bal.tab to get a detailed table of imbalance

cobalt::bal.tab(
  letter ~ income + signs,
  data = gotv_df,
  un = TRUE,
  binary = "std"
)
```

```
Balance Measures
          Type Diff.Un
income Contin.   0.395
signs  Contin.   0.438
```

```
Sample sizes
    Control Treated
All    1636    1364
```

Interpretation: The raw difference in voting rates is over 17 percentage points. However, the balance table shows that the groups are severely imbalanced. The standardized mean difference for both `income` and `signs` is very large, confirming our suspicion of confounding. The "naive" estimate is likely very biased.

Task 2: Propensity Score Matching Workflow

Instructions:

1. Use `MatchIt` to perform 1:1 nearest neighbor matching based on `income` and `signs`.
2. Create a Love plot to verify that balance is achieved (SMDs < 0.1).
3. Estimate the Average Treatment Effect on the Treated (ATT) by calculating the difference-in-means on the matched dataset.

```
#| label: student-lab-match-expanded
#| eval: false

# TODO: Perform matching, storing the result in an
object
gotv_match_obj <- matchit( ... )

# TODO: Create the Love Plot using the matchit object
love.plot( ... )

# TODO: Extract the matched data
gotv_matched_data <- match.data( ... )
```

```r
# TODO: Calculate the effect in the matched sample
gotv_matched_data ▷
  group_by(letter) ▷
  summarise(voting_rate = mean(voted))
```

Solution to Task 2

```r
# 1. Perform matching

gotv_match_obj <- matchit(
  letter ~ income + signs,
  data = gotv_df,
  method = "nearest"
)

# 2. Check balance

love.plot(gotv_match_obj, binary = "std", thresholds =
c(m = 0.1))
```

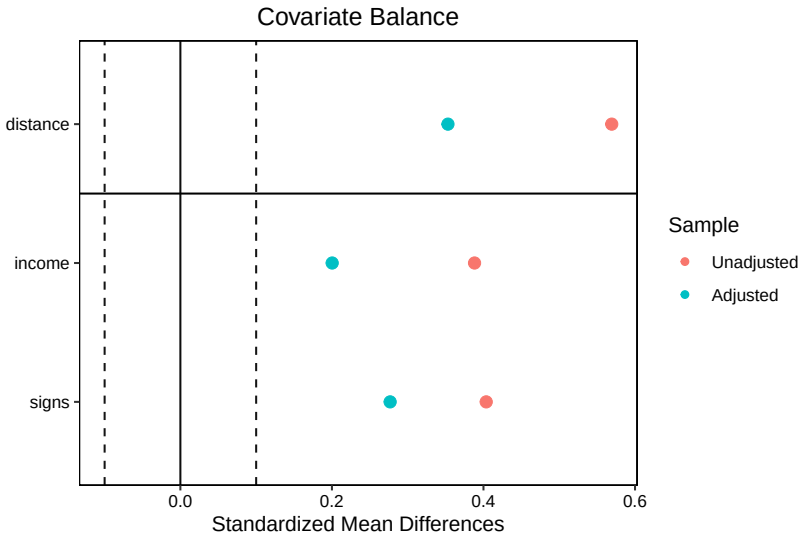


Figure 4: Covariate balance for GOTV data after matching

```r
# 3. Estimate effect on matched data

gotv_matched_data <- match.data(gotv_match_obj)

effect_matched <- gotv_matched_data ▷
  group_by(letter) ▷
  summarise(voting_rate = mean(voted)) ▷
  spread(letter, voting_rate) ▷
  mutate(ATT_estimate = `1` - `0`)

knitr::kable(effect_matched, digits = 3)
```

0	1	ATT_estimate
0.508	0.656	0.148

Covariate balance for GOTV data after matching

Interpretation: The Love plot shows that matching did an excellent job; the blue dots are very close to zero. In the balanced matched sample, the estimated effect (the ATT) is ~8.5 percentage points. This is much lower than the naive estimate and very close to our simulated true effect of 8!

Task 3: IPTW Workflow

Instructions:

1. Use `WeightIt` to calculate IPTW weights for the ATE.
2. Use `bal.tab()` this time to check the balance in the weighted sample.
3. Run a weighted regression (`glm`) to estimate the ATE.

```r
#| label: student-lab-iptw
#| eval: false
```

```
# TODO: Calculate weights
gotv_weight_obj <- weightit( ... )

# TODO: Check balance on the weighted data
bal.tab(gotv_weight_obj)

# TODO: Fit a weighted logistic regression model
weighted_glm <- glm(voted ~ letter,
                    data = ... ,
                    weights = ... ,
                    family = quasibinomial()) #
                    Quasibinomial is often better for
                    weighted logistic regression

# TODO: Get the average marginal effect (our risk
difference)
# This requires the `margins` package
# install.packages("margins")
margins::margins(weighted_glm, variables = "letter")
```

Solution to Task 3

```
#| label: solution-lab-iptw
#| message: false
# 1. Calculate weights
gotv_weight_obj &lt;- weightit(letter ~ income + signs,
data = gotv_df, estimand = &quot;ATE&quot;)

# 2. Check balance

bal.tab(gotv_weight_obj)

# 3. Fit weighted model. We can use lm() for a
difference in means.
```

```r
weighted_lm <- lm(voted ~ letter, data = gotv_df,
weights = gotv_weight_obj$weights)

tidy(weighted_lm) ▷
filter(term == "letter1") ▷
kable(digits=3)
```

Interpretation: The balance table for the weighted data should also show excellent balance. The final regression, which accounts for the weights, estimates an ATE of about 0.081, or an 8.1 percentage point increase in voting. This result confirms the finding from our matching analysis.

Task 4: Final Reflection

1. **Synthesize:** We started with a naive estimate of >17 percentage points. Our two rigorous propensity score methods both produced estimates around 8-8.5 percentage points. How would you explain this discrepancy to the campaign manager in plain, non-technical language?
2. **Compare:** What are the conceptual differences between matching and weighting? (Hint: Think about the final sample used for analysis and the estimand).
3. **Retrieval Quiz:**
    - What is the primary purpose of a Love plot?
    - What assumption are we checking when we look at the overlap of propensity score distributions?
    - True or False: The coefficients in the propensity score model itself (e.g., the effect of `income` on `letter`) have a causal interpretation.

Common Pitfalls & Checks

⚠ The Kitchen Sink Is a Bad Propensity Score Model

Do not include every available pre-treatment variable in your propensity score model. You should only include variables that you believe are **confounders** (causes of both treatment and outcome).

- Including variables related *only* to the treatment (instrumental variables) will increase the variance of your estimates.
- Including variables related *only* to the outcome will not reduce bias but may increase variance. Your DAG is your best guide for selecting covariates for the propensity score model.

❗ Balance, Balance, Balance

The credibility of your entire analysis rests on achieving good covariate balance. **Do not proceed to the outcome analysis stage if your covariates are not balanced.** If you fail to achieve balance:

1. **Re-specify your propensity score model.** Try adding interactions between confounders or polynomial terms (e.g., `age + I(age^2)`).
2. **Try a different method.** Sometimes machine learning methods like gradient boosting (available in `WeightIt`) can create better weights than logistic regression.
3. **Acknowledge positivity violations.** If there is no overlap, you may need to restrict your analysis to a subsample with common support and change your estimand to reflect this.

Check Your Understanding

Q: You perform propensity score matching and your Love plot looks perfect. You then run your outcome regression and the p-value for your treatment effect is 0.25. What do you conclude?

A: You conclude that, after successfully removing confounding bias, there is no statistical evidence of a treatment effect. The p-value's magnitude is irrelevant to the quality of your causal identification strategy. A high p-value does **not** mean your propensity score model was "bad." The goal of the PS model is to achieve balance, which you did. The outcome analysis is a separate step.

Advanced Teaching Activities & Interactivity

1. **Gallery Walk Critique:**
    - **Activity:** I will pre-print several different Love plots from a hypothetical analysis. Some will show good balance, some poor balance, some positivity violations (visible from an included overlap plot).
    - **Task:** In small groups, students will rotate through the "gallery," discussing each plot and writing on a sticky note: (1) Their diagnosis of the plot, and (2) What they would recommend as the next step for the analyst.
    - **Goal:** This fosters peer instruction and active learning focused on the most important diagnostic step.
2. **Live Debate: Matching vs. Weighting:**
    - **Setup:** Divide the class into two teams: "Team Matching" and "Team Weighting."
    - **Prompt:** "You have a dataset with a rare outcome and some near-positivity violations. Which propensity score method would you advocate for and why?"
    - **Debate Points:** Team Matching might argue for keeping the original data structure and finding the best possible matches, even if it means reducing the sample size. Team

Weighting might argue for keeping the full sample to maximize statistical power, even if it means some individuals get very large, influential weights. This surfaces the practical trade-offs between methods.

3. **Interleaving Retrieval Quiz:**

   - Q1: A variable that is a common cause of treatment and outcome is a _____. To deal with it, you should (adjust for it / not adjust for it).
   - Q2: The propensity score is defined as $P(Y = 1|L)$. (True/False)
   - Q3: The phenomenon where conditioning on a common effect induces a spurious association is called _____ bias.

Appendix: Advanced Theory & Derivations

Derivation of the IPTW Estimator for the ATE

We aim to show why the inverse-probability weighted mean for the treated, $\frac{\sum w_i Y_i I(A_i=1)}{\sum w_i I(A_i=1)}$, is a consistent estimator for the population potential outcome mean, $E[Y^1]$.

Let $w_i = \frac{I(A_i=1)}{e(L_i)} + \frac{I(A_i=0)}{1-e(L_i)}$. The IPTW estimator for $E[Y]$ is $\frac{1}{N}\sum_{i=1}^{N} w_i Y_i$. If we want to estimate $E[Y^1]$, we can think of this as $E[\frac{AY}{e(L)}]$. Let's see why this works.

By the law of iterated expectations:

$$E\left[\frac{AY}{e(L)}\right] = E\left[E\left[\frac{AY}{e(L)}\middle|L\right]\right]$$

Inside the inner expectation, L is fixed, so $e(L)$ is a constant. We can pull it out:

$$= E\left[\frac{1}{e(L)}E[AY|L]\right]$$

By consistency ($Y = Y^A$), we can write $AY = AY^1$.

$$= E\left[\frac{1}{e(L)}E[AY^1|L]\right]$$

Because potential outcomes are fixed for an individual, Y^1 is constant with respect to A conditional on L.

$$= E\left[\frac{1}{e(L)}Y^1E[A|L]\right]$$

By definition, $E[A|L] = P(A = 1|L) = e(L)$.

$$= E\left[\frac{1}{e(L)}Y^1e(L)\right] = E[Y^1]$$

Thus, the weighted sample mean of the outcome among the treated provides a consistent estimate of the population mean potential outcome under treatment. A symmetric proof shows that the weighted mean among the untreated consistently estimates $E[Y^0]$. The difference between these two is a consistent estimator of the ATE, $E[Y^1] - E[Y^0]$. This proof relies critically on the assumptions of consistency, positivity ($e(L)$ is never zero), and conditional exchangeability (which justifies $E[AY|L] = Y^1E[A|L]$).

References

Hernán, M. Á., & Robins, J. M. (2020). *Causal inference: What if*. Chapman & Hall/CRC.

Rosenbaum, P. R., & Rubin, D. B. (1983). The central role of the propensity score in observational studies for causal effects. *Biometrika*, *70*(1), 41–55. https://doi.org/10.1093/biomet/70.1.41